

localStorageとは？ Webページに情報を保存する仕組み

このドキュメントでは、Webブラウザの機能である `localStorage` について解説します。

`localStorage` を使うと、Webページを閉じたり、コンピュータを再起動したりしても、作成したデータをユーザーのコンピュータに保存しておくことができます。

1. localStorageの概要

`localStorage` は、HTML5から導入されたWeb Storage APIの一つで、キーと値のペアでデータを永続的に保存するための仕組みです。

- **永続性** : ブラウザを閉じてもデータは消えません。ユーザーが明示的に削除するまで残ります。
- **保存場所** : ユーザーのコンピュータのブラウザ内。サーバーには送信されません。
- **データ形式** : 文字列のみ。オブジェクトや配列を保存する場合は、JSON形式に変換する必要があります。
- **容量** : 通常、オリジン（ドメイン）ごとに5MB～10MB程度。大量のデータ保存には向きません。
- **同期処理** : `localStorage` の操作は同期的に行われます。複雑な処理や大量のデータアクセスは、ページのパフォーマンスに影響を与える可能性があります。

2. なぜlocalStorageを使うのか？

動的なWebサイト（例：前の演習で作成した「やること動的サイト」）では、ページをリロードすると追加したタスクが消えてしまいました。これは、データがメモリ上に一時的に保存されているだけで、永続化されていなかったためです。

`localStorage` を使うことで、以下のようなメリットがあります。

- **ユーザー体験の向上** :
 - 入力途中のフォームの内容を一時保存し、後で再開できるようにする。（例：長文のブログ記事作成画面）
 - Webアプリケーションの設定（例：テーマカラー、表示件数、言語設定）をユーザーごとに保存する。
 - 今回の演習のように、To-Doリストのタスクや進捗状況を保存する。
 - ECサイトで最近見た商品を記録しておく。
- **オフライン機能の基礎** : 限定的ながら、オフラインでも利用可能な機能の実現に役立ちます。（例：一度読み込んだ記事をオフラインで閲覧可能にする）

- **パフォーマンス改善**：サーバーへのリクエスト回数を減らすことで、表示速度の向上やサーバー負荷の軽減に繋がる場合があります。（例：頻繁にアクセスするが変更の少ないデータをキャッシュする）

3. localStorageの基本的な使い方

`localStorage` は、グローバルな `window` オブジェクトのプロパティとして提供されており、`localStorage` または `window.localStorage` でアクセスできます。

主なメソッドは以下の通りです。

- **データの保存**：`localStorage.setItem(key, value)`

- `key`：保存するデータの名前（文字列）
- `value`：保存するデータ（文字列）

```
localStorage.setItem('username', 'John Doe');  
localStorage.setItem('isLoggedIn', 'true');
```

- **データの取得**：`localStorage.getItem(key)`

- `key`：取得するデータの名前（文字列）
- データが存在しない場合は `null` を返します。

```
let username = localStorage.getItem('username'); // "John Doe"  
let theme = localStorage.getItem('theme');      // null（もしthemeが未設定なら）
```

- **データの削除**：`localStorage.removeItem(key)`

- `key`：削除するデータの名前（文字列）

```
localStorage.removeItem('isLoggedIn');
```

- **すべてのデータを削除**：`localStorage.clear()`

- 現在のオリジン（ドメイン）の `localStorage` に保存されているすべてのデータを削除します。

```
localStorage.clear();
```

- **保存されているキーの取得**：`localStorage.key(index)`

- `index`：取得したいキーのインデックス番号（0から始まる整数）

- `localStorage` に保存されているキーの数を `localStorage.length` で取得できます。

```
for (let i = 0; i < localStorage.length; i++) {  
  let key = localStorage.key(i);  
  let value = localStorage.getItem(key);  
  console.log(`Key: ${key}, Value: ${value}`);  
}
```

4. オブジェクトや配列の保存

`localStorage` は文字列しか保存できないため、JavaScriptのオブジェクトや配列をそのまま保存しようとすると、意図しない結果（例: `[object Object]`）になります。

オブジェクトや配列を保存・取得するには、`JSON.stringify()` と `JSON.parse()` を使って、JSON文字列に変換する必要があります。

- **保存時** : `JSON.stringify()` でオブジェクトをJSON文字列に変換

```
const user = { name: 'Alice', age: 30 };  
localStorage.setItem('userData', JSON.stringify(user));
```

- **取得時** : `JSON.parse()` でJSON文字列をオブジェクトに復元

```
const storedUserJSON = localStorage.getItem('userData');  
if (storedUserJSON) {  
  const userObject = JSON.parse(storedUserJSON);  
  console.log(userObject.name); // "Alice"  
}
```

5. 注意点とベストプラクティス

- **機密情報を保存しない** : `localStorage` はクライアントサイドで簡単にアクセスできるため、パスワードや個人情報などの機密性の高い情報を保存するのには適していません。
- **容量制限を意識する** : 保存できる容量には限りがあります（通常5MB～10MB）。大きなデータを保存するのは避けましょう。容量を超えると `QuotaExceededError` が発生します。
- **エラーハンドリング** :
 - `setItem` 時にクォータ超過エラー（`QuotaExceededError`）が発生する可能性があります。

- `getItem` で取得した値が期待する形式でない場合や、`JSON.parse()` で不正なJSON文字列をパースしようとした場合にエラーが発生する可能性があります。
- 必要に応じて `try...catch` ブロックでエラー処理を実装しましょう。

```
// 例: JSON.parse時のエラーハンドリング
try {
  const data = JSON.parse(localStorage.getItem('myData'));
  if (data) {
    // データを使った処理
  }
} catch (error) {
  console.error('localStorageからのデータ読み込みまたは解析に失敗しました:', error);
  // エラー時のフォールバック処理
}
```

- **キーの命名規則** : 複数のアプリケーションやスクリプトが同じオリジンで動作する場合、キー名が衝突しないように注意が必要です。プレフィックスを付ける（例: `myapp_username`）などの工夫をしましょう。
- **同期処理の影響** : 大量のデータを一度に読み書きすると、ブラウザのメインスレッドをブロックし、ページの応答性が低下する（画面が固まるなど）可能性があります。特に複雑な処理を伴う場合は注意が必要です。
- **Session Storageとの違い** :
 - `localStorage` : データは永続的。ブラウザを閉じても消えない。
 - `sessionStorage` : データはセッション限り。ブラウザのタブやウィンドウを閉じると消える。
 - 使い分けの例 : `localStorage` はユーザー設定など長期的に保持したい情報に、`sessionStorage` は入力フォームの一次的な値など、そのセッション中だけ必要な情報に適しています。

6. 開発者ツールでの確認方法

ほとんどのモダンブラウザには、`localStorage` の内容を確認・編集できる開発者ツールが搭載されています。

1. Webページ上で右クリックし、「検証」や「要素を調査」などを選択して開発者ツールを開きます。
2. 「アプリケーション」(Application) タブ（または「ストレージ」(Storage) タブなど、ブラウザによって名称が若干異なります）を選択します。
3. 左側のメニューから「ローカルストレージ」(Local Storage) を展開し、対象のドメインを選択します。
4. 保存されているキーと値の一覧が表示され、ここから値の確認、編集、削除が可能です。

（ここに開発者ツールのスクリーンショット画像を挿入することを推奨します。例: Chrome開発者ツールのlocalStorage表示画面）

まとめ

`localStorage` は、Webページに手軽にデータを永続化できる便利な機能です。今回の演習では、この `localStorage` を使って、To-Doリストのタスクを保存し、ページをリロードしてもタスクが消えないように改良していきます。

次のステップでは、実際にこの `localStorage` を使って、タスクを保存する機能を実装してみましょう。